



Session 07:

TypeScript Essentials: TypeScript Core & OOP

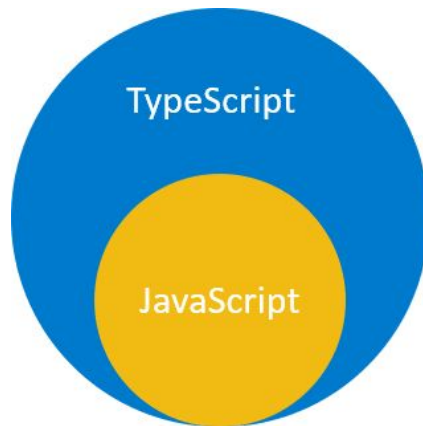


1. **Biết và hiểu khái niệm về TypeScript**
2. **Hiểu và sử dụng thành thạo các kiểu dữ liệu cơ bản**
3. **Áp dụng được (Union, Intersection, Enum, Tuple)**
4. **Hiểu 4 tính chất lập trình hướng đối tượng (OOP)**
5. **Hiểu khái niệm tham số kiểu Generic trong Function, Class, Interface**

1. Khái niệm TypeScript

TypeScript là một ngôn ngữ lập trình được phát triển bởi Microsoft

- Được coi là một phiên bản nâng cao của JavaScript
- Có thể được sử dụng để phát triển các ứng dụng chạy ở client-side (Angular2) và server-side (NodeJS)



So sánh JavaScript và TypeScript

	JavaScript	TypeScript
Bản chất	Kiểu động (Dynamic), gán dữ liệu tự do	Kiểu tĩnh (Static), khai báo rõ ràng
Phát hiện lỗi	Lúc chạy chương trình (Runtime)	Ngay khi viết code (Compile-time)
Hỗ trợ IDE	Gợi ý cơ bản	Gợi ý thông minh, refactor an toàn, bắt lỗi tức thời
Ứng dụng	Dự án nhỏ, script đơn giản	Dự án quy mô lớn, làm việc nhóm

Cài đặt Node.js và npm

Node.js là nền tảng cần thiết để chạy các công cụ

Tải và cài đặt [Node.js](https://nodejs.org/)

- **Bước 1:** Truy cập và download tại trang chính thức: <https://nodejs.org/>
- **Bước 2:** Kiểm tra cài đặt thành công: **node -v**



Cài đặt TypeScript

Cài TypeScript compiler:

- **Bước 1:** Gõ trong terminal : **npm install -g typescript**
- **Bước 2:** Để kiểm tra đã cài thành công : **tsc -v**



Biên dịch Typescript code

Bước 1 : Sử dụng lệnh **npm init -y** để tạo file package.json

```
PS D:\TailieuHocTypescript\Typescript> npm init -y  
Wrote to D:\TailieuHocTypescript\Typescript\package.json:
```

```
{  
  "name": "typescript",  
  "version": "1.0.0",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "description": ""  
}
```

Bước 2 : Sử dụng lệnh **tsc --init** để tạo file tsconfig

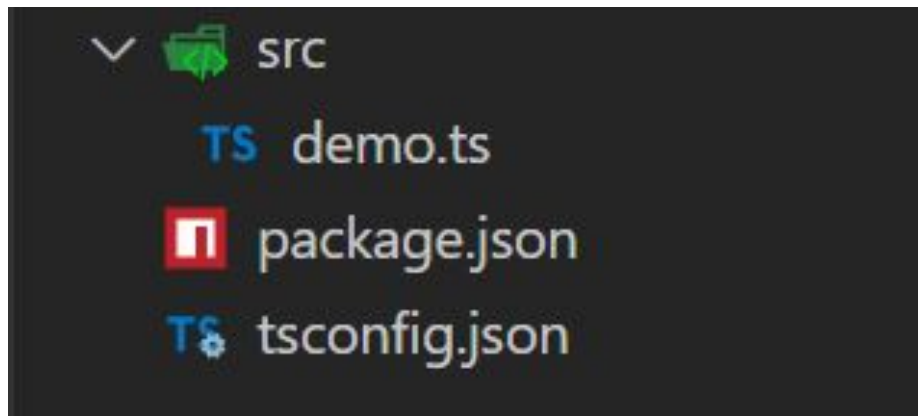
```
PS D:\TailieuHocTypescript\Typescript> tsc --init
```

Created a new tsconfig.json with:

```
target: es2016  
module: commonjs  
strict: true  
esModuleInterop: true  
skipLibCheck: true  
forceConsistentCasingInFileNames: true
```

Biên dịch Typescript code

Bước 3 : Tạo Folder src để chứa file Typescript



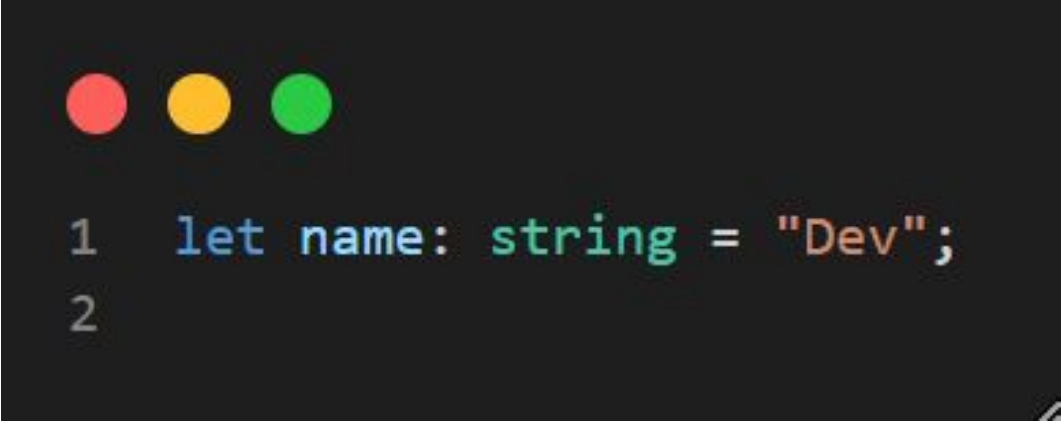
Biên dịch Typescript code

Bước 4 : Config trong file tsconfig.json

```
D: > TaiLieuHocTypescript > Typescript > tsconfig.json > ...
1  {
2      "compilerOptions": {
3          /* Language and Environment */
4          "target": "es2016", // Xác định phiên bản JavaScript mà TypeScript sẽ biên dịch xuống
5          "lib": ["es6", "DOM"], //Xác định các thư viện chuẩn (standard library) mà TypeScript sẽ dùng để kiểm tra kiểu
6          "module": "commonjs", //Chỉ định hệ thống module nào sẽ được dùng để biên dịch
7          "rootDir": "./src", // Chỉ định thư mục gốc chứa mã nguồn TypeScript
8          "noEmit": false, //Chỉ định có biên dịch file .ts ra file .js hay không
9          "outDir": "./dist", //Xác định thư mục đầu ra chứa file .js sau khi biên dịch
10         "noEmitOnError": true /* Disable emitting files if any type checking errors are reported. */,
11         "esModuleInterop": true /* Emit additional JavaScript to ease support for importing CommonJS modules. This enables 'allowSyntheticDefaultImports' */
12         "forceConsistentCasingInFileNames": true /* Ensure that casing is correct in imports. */,
13         "strict": true /* Enable all strict type-checking options. */,
14         "skipLibCheck": true /* Skip type checking all .d.ts files. */
15     },
16     "include": ["src/**/*"]
17 }
18
```

Biên dịch Typescript code

Bước 5 : Code demo.ts

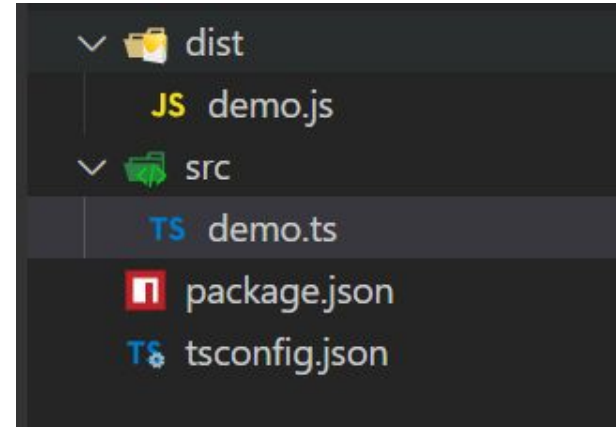
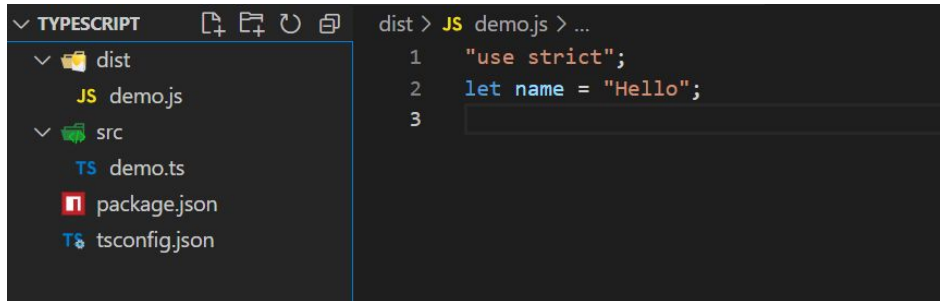


```
1  let name: string = "Dev";  
2
```

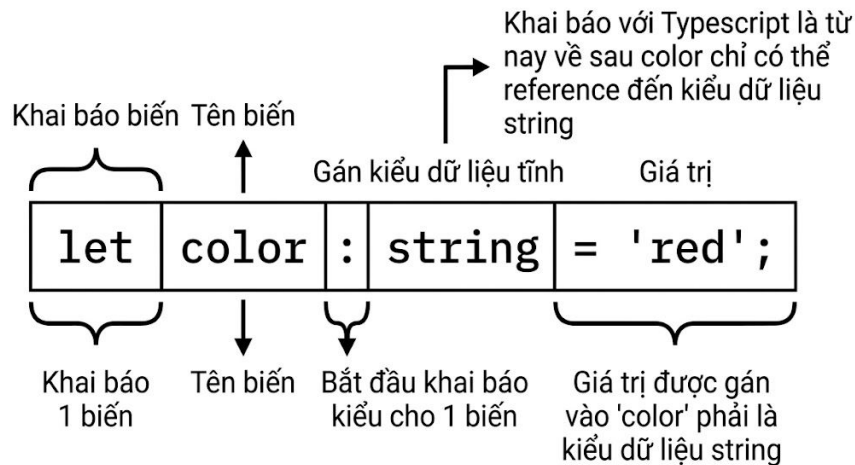
Biên dịch Typescript code

Bước 6 : Sau khi code xong khi muốn build tất cả code typescript sang javascript theo cấu hình trong tsconfig.json, bạn đơn giản là chạy lệnh sau: tsc

```
PS D:\TaiLieuHocTypescript\Typescript> tsc
```



Biến và các kiểu dữ liệu cơ bản trong Typescript



```
let city: string = 'Hà Nội';
```

```
1 let city: string = 'Hà Nội';  
2 city = 20; |  
Type 'number' is not assignable to type  
'string'. ts(2322)  
Error (TS2322)
```

Kiểu dữ liệu tham chiếu Array

Mảng trong TypeScript yêu cầu sự đồng nhất, tập hợp các phần tử phải có cùng một kiểu dữ liệu.

Cách 1: Cú pháp ngắn gọn

```
let listNumber: Number[] = [1, 2, 3, 4, 5];  
let listName: string[] = ["Phong", "Linh"];
```

Cách 2: Cú pháp Generic

```
let flags: Array<boolean> = [true, false, true];
```

Bảo vệ tính đồng nhất

```
let listNumber: Number[] = [1, 2, 3, 4, 5];  
listNumber.push('A');
```

Argument of type 'string' is not assignable to parameter of type 'Number'.ts(2345)

Error (TS2345)

Kiểu dữ liệu tham chiếu Object

Tạo đối tượng

```
let teacher: { name: string; age: number } = {  
  name: "Tommy",  
  age: 40  
};
```

Lỗi tạo mới

```
teacher.phone = 12345;  
Property 'phone' does not exist on type '{  
  name: string; age: number; }'. ts(2339)  
Error (TS2339)
```

Lỗi cập nhật

```
teacher.age = "forty";  
Type 'string' is not assignable to type  
'number'. ts(2322)  
Error (TS2322)
```

Interface

Khi xác định các điểm đặc trưng của 1 đối tượng, ta có thể sử dụng Interface

```
interface Person {  
    name: string;  
    age: number;  
}  
  
let teacher: Person = {  
    name: "Tommy",  
    age: 40,  
};
```

Kiểu kết hợp (Union Type)

Toán tử “|” cho phép một biến nhận một trong nhiều kiểu dữ liệu khác nhau



```
let id: string | number;
```



```
id = 'DEV_01'; // Hợp lệ (string)
```



```
id = 12345; // Hợp lệ (number)
```

```
id = true; // Error
```

```
Type 'boolean' is not assignable to type  
'string | number'.ts(2322)
```

△ Error (TS2322) [icon] [icon] [icon] [icon]

Type Alias

Đặt tên tùy chỉnh cho một kiểu dữ liệu. Giúp mã nguồn dễ đọc, dễ hiểu, dễ bảo trì hơn

Lập code

```
let userId: string | number;  
let orderId: string | number;  
let transactionId: string | number;
```

Tối ưu
hóa

Gọn gàng

```
type ID = string | number;  
  
let userId: ID;  
let orderId: ID;  
let transactionId: ID;
```

Intersection Type

Toán tử “&” gộp tất cả thuộc tính từ nhiều kiểu thành phần vào một kiểu duy nhất

```
interface HasName {  
    name: string;  
}  
  
interface HasAge {  
    age: number;  
}  
  
type Person = HasName & HasAge;
```

So sánh Type Alias và Interface

Tiêu chí	Interface	Type Alias
Cú pháp khai báo	interface Name { ... }	type Name = ...
Khả năng kế thừa	Dùng từ khóa extends	Dùng Intersection (&)
Gộp khai báo (Declaration Merging)	Tự động gộp nếu trùng tên	Báo lỗi trùng lặp
Định nghĩa Union, Tuple, Primitive	Chỉ dùng cho Object	Rất mạnh mẽ

Kiểu dữ liệu đặc biệt

void (Không trả về giá trị)

```
const log = (msg: string): void => {  
    console.log(msg);  
    return msg;  
}  
Type 'string' is not assignable to type  
'void'. ts(2322)  
Error (TS2322)
```

any (Bỏ qua kiểm tra kiểu)

```
let value: any = "hello";  
value = 42; // TypeScript bỏ qua  
value = true; // Dễ gây lỗi Runtime
```

unknown (Phiên bản an toàn của any)

```
let value: any = "hello";  
if (typeof value == "string") {  
    console.log(value);  
}
```

never (không bao giờ kết thúc)

```
const throwError = (): never => {  
    throw new Error("Lỗi hệ thống!");  
};
```

Cấu trúc chặt chẽ: Enum & Tuple

Enum (Tập hợp hằng số)

```
enum Status {  
    Pending,  
    Success,  
    Failed  
}  
let currentStatus: Status = Status.Success  
;
```

Nhóm các trạng thái liên quan để code, dễ đọc, tránh hand-code sai

Tuple (Mảng cố định)

```
let point: [number, number] = [10, 20];
```

```
point = [10, 20, 30];  
Type '[number, number, number]' is not  
assignable to type '[number, number]'.  
Source has 3 element(s) but target allows  
only 2. ts(2322)
```

△ Error (TS2322) | 📄 | ⚙️ | 🗑️ | 🔗

Mảng bị khóa cứng số lượng phần tử và kiểu dữ liệu tại từng vị trí

Lập trình hướng đối tượng OOP

Là một mô hình lập trình giúp tổ chức mã nguồn thành các **"đối tượng"** mô phỏng thực thể ngoài đời thực, giúp dễ bảo trì, mở rộng và tái sử dụng code



Class và các thành phần của lớp

Class trong TypeScript là bản thiết kế (template) dùng để tạo đối tượng (object), định nghĩa các thuộc tính (attribute) và phương thức (method) chung cho các đối tượng cùng loại

Thành phần:

- Thuộc tính (Dữ liệu / Trạng thái)
- Phương thức (Hành vi / Logic)

Cú pháp:

- Dùng từ khóa class, hàm khởi tạo constructor để gán giá trị ban đầu, và từ khóa this để trỏ đến đối tượng hiện tại

```
class Animal {  
  name: string;  
  constructor(name: string) {  
    this.name = name;  
  }  
  speak() {  
    console.log(`${this.name}  
make a sound`);  
  }  
}  
  
const dog = new Animal("Dog");  
dog.speak();
```

Access Modifier

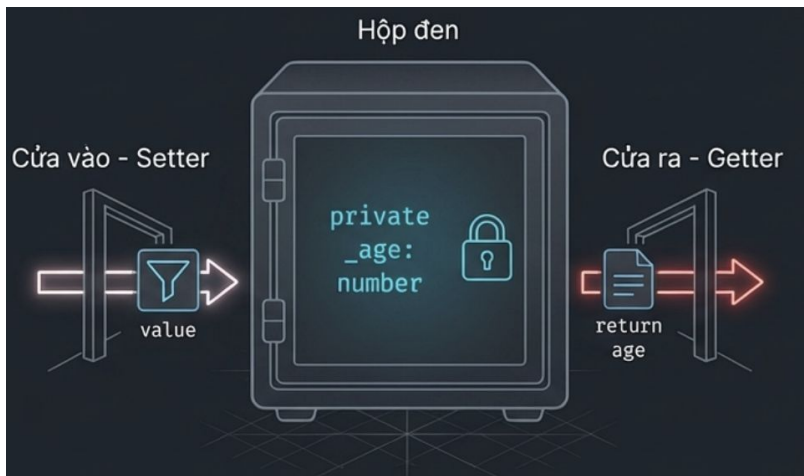
Là từ khóa giúp kiểm soát quyền truy cập đến các thuộc tính và phương thức của một lớp. TypeScript cung cấp ba loại modifier chính:

- **public** – Mặc định: Có thể truy cập ở mọi nơi
- **private**: Chỉ truy cập được bên trong chính lớp đó
- **protected**: Truy cập được trong lớp đó và các lớp con kế thừa

```
class Person {  
  public name: string;  
  protected email: string;  
  readonly id: number;  
  
  constructor(name: string, email:  
string, id: number) {  
    this.name = name;  
    this.email = email;  
    this.id = id;  
  }  
}
```

Tính đóng gói (Encapsulation)

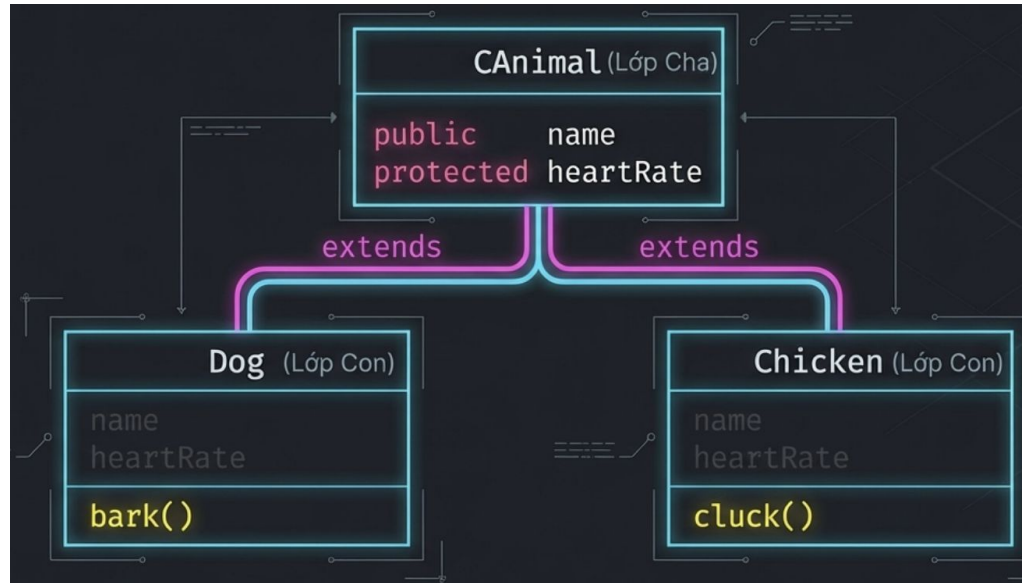
Không cho bên ngoài chạm trực tiếp vào dữ liệu. Mọi giao tiếp phải thông qua **getter/setter** để kiểm tra tính hợp lệ



```
class User {  
  private _age: number;  
  constructor(age: number) {  
    this._age = age;  
  }  
  get getAge(): number {  
    return this._age;  
  }  
  set setAge(value: number) {  
    if (value > 0 && value < 150) {  
      this._age = value;  
    } else {  
      console.log("Tuổi không hợp lệ");  
    }  
  }  
}
```

Tính kế thừa (Inheritance)

Cho phép một lớp mới (**gọi là lớp con**) được tạo ra dựa trên lớp đã có (**lớp cha**), và tự động thừa hưởng các thuộc tính (**properties**) và phương thức (**methods**) của lớp cha



Cú pháp extends và hàm super()

Từ khóa **extends** dùng để khai báo kế thừa

Lưu ý: Khi lớp con có constructor riêng, nó bắt buộc phải gọi hàm **super()** để "đánh thức" constructor của lớp cha trước khi làm bất cứ việc gì khác

```
class Parent {  
    public name: string;  
    constructor(name: string) {  
        this.name = name;  
    }  
}  
  
class Children extends Parent {  
    constructor(name: string, age: number) {  
        super(name);  
        // Gọi constructor của lớp Parent  
    }  
}
```

Tính đa hình (Polymorphism)

Cho phép lớp con **ghi đè** phương thức của lớp cha để tạo ra hành vi đặc thù, dù gọi chung một tên hàm



```
class Vehicle {  
    move(): void {}  
}  
  
class Car extends Vehicle {  
    move(): void {  
        console.log("Chiếc ô tô đang chạy");  
    }  
}
```

Ghi đè (Override): Đập bỏ logic cũ của lớp cha, triển khai logic mới phù hợp với thực thể con.

Lớp trừu tượng (Abstract class)

Là khuôn mẫu cấp cao, chứa các khung sườn cơ bản. Không thể khởi tạo trực tiếp bằng từ khóa new

```
1  abstract class CShape {
2      abstract getArea(): number;
3      display(): void {
4          console.log('Diện tích : ${this.getArea()}');
5      }
6  }
7
8  class CRectangle extends CShape {
9      constructor(private width: number, private height: number) {
10         super();
11     }
12     getArea(): number {
13         return this.width * this.height;
14     }
15 }
16 class CCircle extends CShape {
17     constructor(private radius: number) {
18         super();
19     }
20     getArea(): number {
21         return Math.PI * this.radius * this.radius;
22     }
23 }
```

Abstract Method

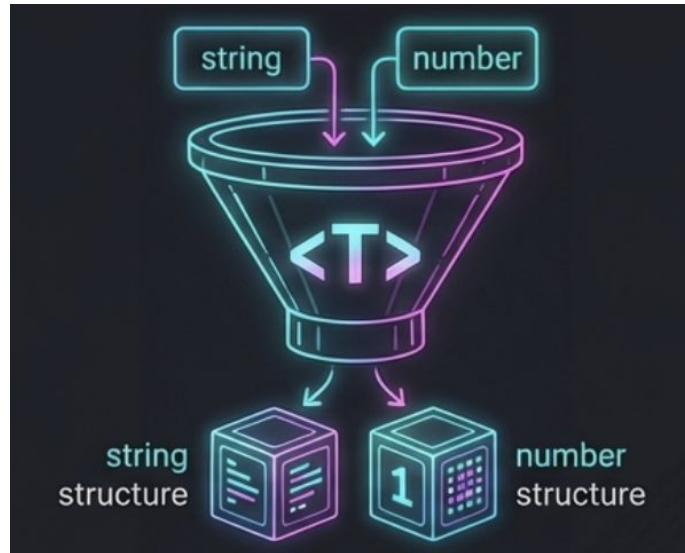
Lớp cha chỉ định nghĩa tên hàm, bắt buộc các lớp con kế thừa phải tự viết phần thân

Chú ý: Không có cặp ngoặc nhọn `{ }`. Lớp con quên triển khai hàm này thì TypeScript sẽ lập tức báo lỗi

```
abstract class ClassName {  
    abstract methodName(params): ReturnType;  
}
```

Generic type

Là tính năng cho phép khai báo tham số kiểu (type parameter) cho hàm, lớp hoặc giao diện (interface), giúp chúng làm việc với nhiều kiểu dữ liệu khác nhau thay vì chỉ một kiểu cố định



Generic Function

Là hàm nhận tham số kiểu động $<T>$ tái sử dụng, an toàn cho mọi kiểu

```
const identity = <T>(arg: T): T => {  
  return arg;  
};
```

Kiểu vào và kiểu ra được
đồng bộ tuyệt đối

```
identity<string>("Hello");  
identity<number>(42);
```

Generic Class

Là lớp có thể làm việc với nhiều kiểu dữ liệu khác nhau thông qua tham số kiểu (type parameter)

```
1 class ClassName<T> {  
2     value: T;  
3     constructor(val: T) {  
4         this.value = val;  
5     }  
6     getValue(): T {  
7         return this.value;  
8     }  
9 }
```

```
1 const numBox = new ClassName<number>(123);  
2 console.log(numBox.getValue()); // 123  
3  
4 const strBox = new ClassName<string>("Hello");  
5 console.log(strBox.getValue()); // Hello
```

Generic interface

Là một giao diện (interface) trong TypeScript có thể làm việc với nhiều kiểu dữ liệu khác nhau bằng cách sử dụng tham số kiểu (type parameter)

```
1 interface IName<T>{  
2     property: T;  
3     method(value: T): void;  
4 }
```

```
1 interface IBox<T> {  
2     value: T;  
3 }  
4  
5 const numberBoxG: IBox<number> = { value: 10 };  
6  
7 console.log(numberBox);  
8
```

- ❑ Hiểu bản chất cốt lõi của TypeScript và biết cách thiết lập, cấu hình môi trường lập trình tối ưu.
- ❑ Sử dụng được các kiểu dữ liệu từ cơ bản đến nâng cao (Union, Intersection, Enum, Tuple) để kiểm soát dữ liệu
- ❑ Áp dụng thành thạo Interface và Type Alias để định nghĩa cấu trúc dữ liệu chặt chẽ và dễ tái sử dụng
- ❑ Hiểu sâu 4 tính chất OOP và sử dụng được Class, Access Modifiers để xây dựng ứng dụng có cấu trúc
- ❑ Biết cách tối ưu hóa hiệu suất mã nguồn và đảm bảo an toàn kiểu dữ liệu thông qua việc sử dụng Generic



HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN